

Introduction to Hadoop and MapReduce

Apache Hadoop

Apache Hadoop

- Scalable fault-tolerant distributed system for Big Data
 - Distributed Data Storage
 - Distributed Data Processing
 - Borrowed concepts/ideas from the systems designed at Google (Google File System for Google's MapReduce)
 - Open source project under the Apache license
 - But there are also many commercial implementations (e.g., Cloudera, Hortonworks, MapR)

Hadoop History

- Dec 2004 – Google published a paper about GFS
- July 2005 – Nutch uses MapReduce
- Feb 2006 – Hadoop becomes a Lucene subproject
- Apr 2007 – Yahoo! runs it on a 1000-node cluster
- Jan 2008 – Hadoop becomes an Apache Top Level Project
- Jul 2008 – Hadoop is tested on a 4000 node cluster

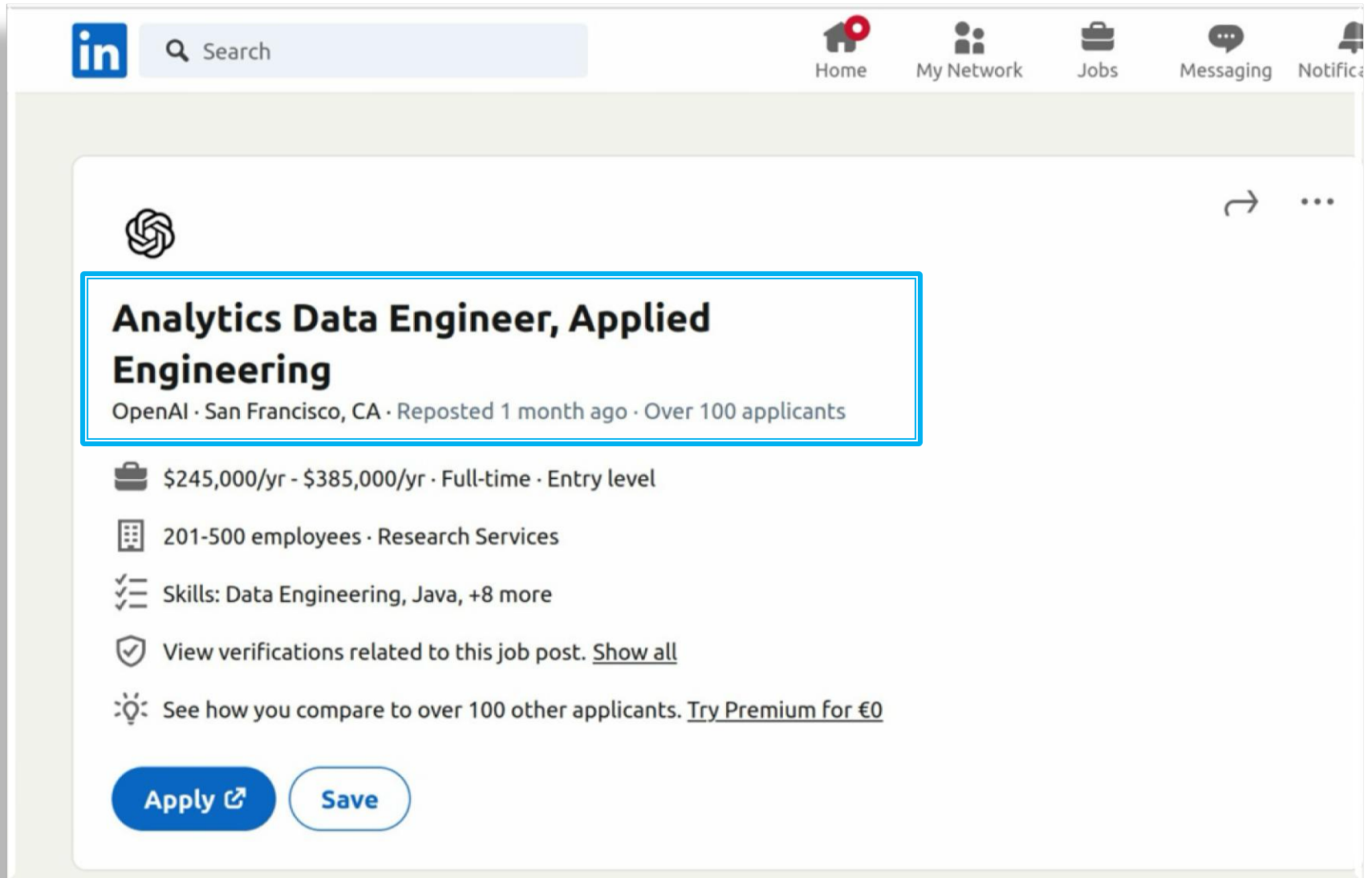
Hadoop History

- Feb 2009 – The Yahoo! Search Webmap is a Hadoop application that runs on more than 10,000 core Linux cluster
- June 2009 – Yahoo! made available the source code of its production version of Hadoop
- In 2010 Facebook claimed that they have the largest Hadoop cluster in the world with 21 PB of storage
 - On July 27, 2011 they announced the data has grown to 30 PB.

Who uses/used Hadoop?

- Amazon
- Facebook
- Google
- IBM
- Joost
- Last.fm
- New York Times
- PowerSet
- Veoh
- Yahoo!
- OpenAI
-

Job posted by OpenAI



The screenshot shows a LinkedIn job post interface. At the top, there is a navigation bar with the LinkedIn logo, a search bar, and icons for Home, My Network, Jobs, Messaging, and Notifications. Below the navigation bar, the job post is displayed. It features the OpenAI logo, the job title "Analytics Data Engineer, Applied Engineering" (highlighted with a blue border), and the company name "OpenAI" along with location "San Francisco, CA", reposted date "1 month ago", and number of applicants "Over 100 applicants". Below the job title, there are several details: a salary range of "\$245,000/yr - \$385,000/yr", a full-time entry-level position, 201-500 employees in Research Services, and skills including Data Engineering, Java, and 8 more. There are also links to view verifications and to see how the user compares to other applicants. At the bottom, there are "Apply" and "Save" buttons.

 Search

Home My Network Jobs Messaging Notifications



Analytics Data Engineer, Applied Engineering

OpenAI · San Francisco, CA · Reposted 1 month ago · Over 100 applicants

 \$245,000/yr - \$385,000/yr · Full-time · Entry level

 201-500 employees · Research Services

 Skills: Data Engineering, Java, +8 more

 View verifications related to this job post. [Show all](#)

 See how you compare to over 100 other applicants. [Try Premium for €0](#)

[Apply](#) [Save](#)

Job posted by OpenAI

Analytics Data Engineer, Applied Engineering

OpenAI San Francisco, California, United States

In This Role, You Will

- Design, build and manage our data pipelines, ensuring all user event data is seamlessly integrated into our data warehouse.
- Develop canonical datasets to track key product metrics including user growth, engagement, and revenue.
- Work collaboratively with various teams, including, Infrastructure, Data Science, Product, Marketing, Finance, and Research to understand their data needs and provide solutions.
- Implement robust and fault-tolerant systems for data ingestion and processing.
- Participate in data architecture and engineering decisions, bringing your strong experience and knowledge to bear.
- Ensure the security, integrity, and compliance of data according to industry and company standards.

You Might Thrive In This Role If You

- Have 3+ years of experience as a data engineer and 8+ years of any software engineering experience(including data engineering).
- Proficiency in at least one programming language commonly used within Data Engineering, such as Python, Scala, or Java.
- Experience with distributed processing technologies and frameworks, such as Hadoop, Flink and distributed storage systems (e.g., HDFS, S3).
- Expertise with any of ETL schedulers such as Airflow, Dagster, Prefect or similar frameworks.
- Solid understanding of Spark and ability to write, debug and optimize Spark code.

Job posted by OpenAI

Analytics Data Engineer, Applied Engineering

OpenAI San Francisco, California, United States

In This Role, You Will

- Design, build and manage our data pipelines, ensuring all user event data is seamlessly integrated into our data warehouse.
- Develop canonical datasets to track key product metrics including user growth, engagement, and revenue.
- Work collaboratively with various teams, including, Infrastructure, Data Science, Product, Marketing, Finance, and Research to understand their data needs and provide solutions.
- Implement robust and fault-tolerant systems for data ingestion and processing.
- Participate in data architecture and engineering decisions, bringing your strong experience and knowledge to bear.
- Ensure the security, integrity, and compliance of data according to industry and company standards.

You Might Thrive In This Role If You

- Have 3+ years of experience as a data engineer and 8+ years of any software engineering experience(including data engineering).
- Proficiency in at least one programming language commonly used within Data Engineering, such as Python, Scala, or Java.
- Experience with distributed processing technologies and frameworks, such as Hadoop, Flink and distributed storage systems (e.g., HDFS, S3).
- Expertise with any of ETL schedulers such as Airflow, Dagster, Prefect or similar frameworks.
- Solid understanding of Spark and ability to write, debug and optimize Spark code.

Job posted by OpenAI

Analytics Data Engineer, Applied Engineering

OpenAI San Francisco, California, United States

For US Based Candidates: Pursuant to the San Francisco Fair Chance Ordinance, we will consider qualified applicants with arrest and conviction records.

We are committed to providing reasonable accommodations to applicants with disabilities, and requests can be made via this link.

[OpenAI Global Applicant Privacy Policy](#)

At OpenAI, we believe artificial intelligence has the potential to help people solve immense global challenges, and we want the upside of AI to be widely shared. Join us in shaping the future of technology.

Posted on Sep 26, 2023.

[See less ^](#)

Set alert for similar jobs

Data Engineer, San Francisco, CA

 **Set alert**

Hadoop vs. HPC

- **Hadoop**

- Designed for **Data intensive workloads**
- Usually, no CPU demanding/intensive tasks

- **HPC** (High-performance computing)

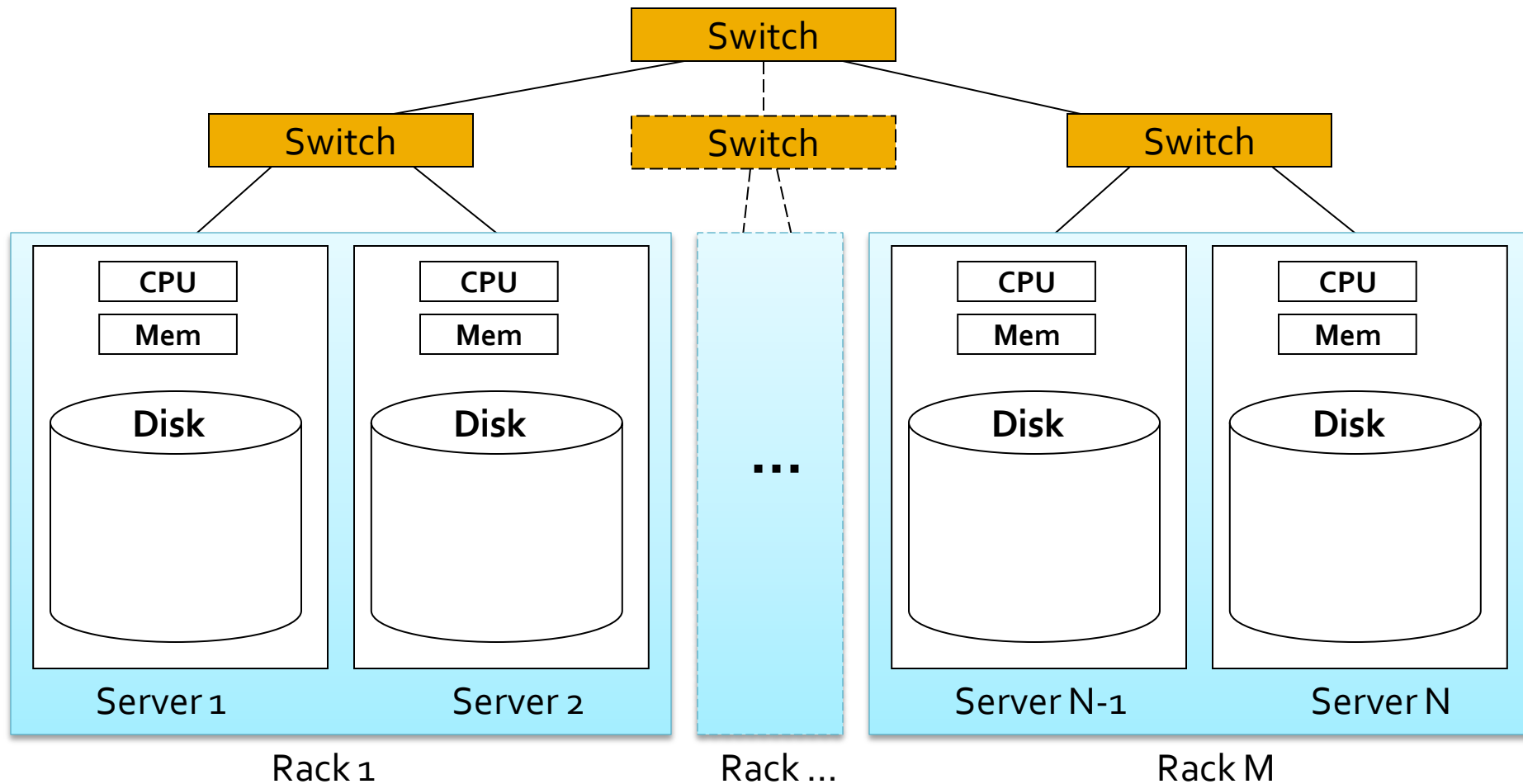
- A supercomputer with a high-level computational capacity
 - Performance of a supercomputer is measured in floating-point operations per second (FLOPS)
- Designed for **CPU intensive tasks**
- Usually it is used to process “small” data sets

Hadoop: main components

↳ *scalable fault-tolerant distributed system designed for Big data*

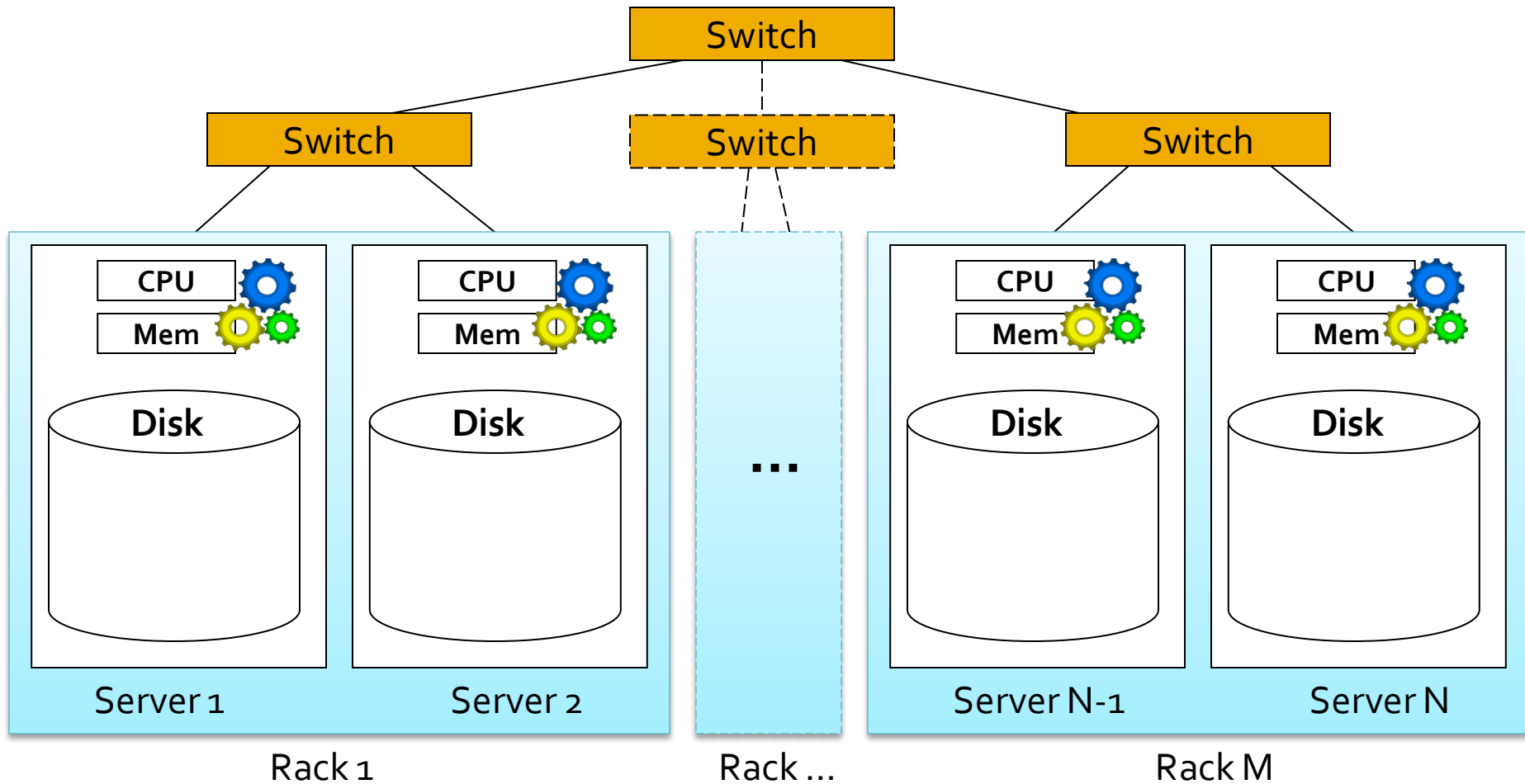
- Core components of Hadoop:
 - Distributed Big Data Processing Infrastructure based on the MapReduce programming paradigm
 - Provides a high-level abstraction view
 - Programmers do not need to care about task scheduling and synchronization
 - Fault-tolerant
 - Node and task failures are automatically managed by the Hadoop system
 - HDFS (Hadoop Distributed File System)
 - High availability distributed storage
 - Fault-tolerant

Hadoop: main components



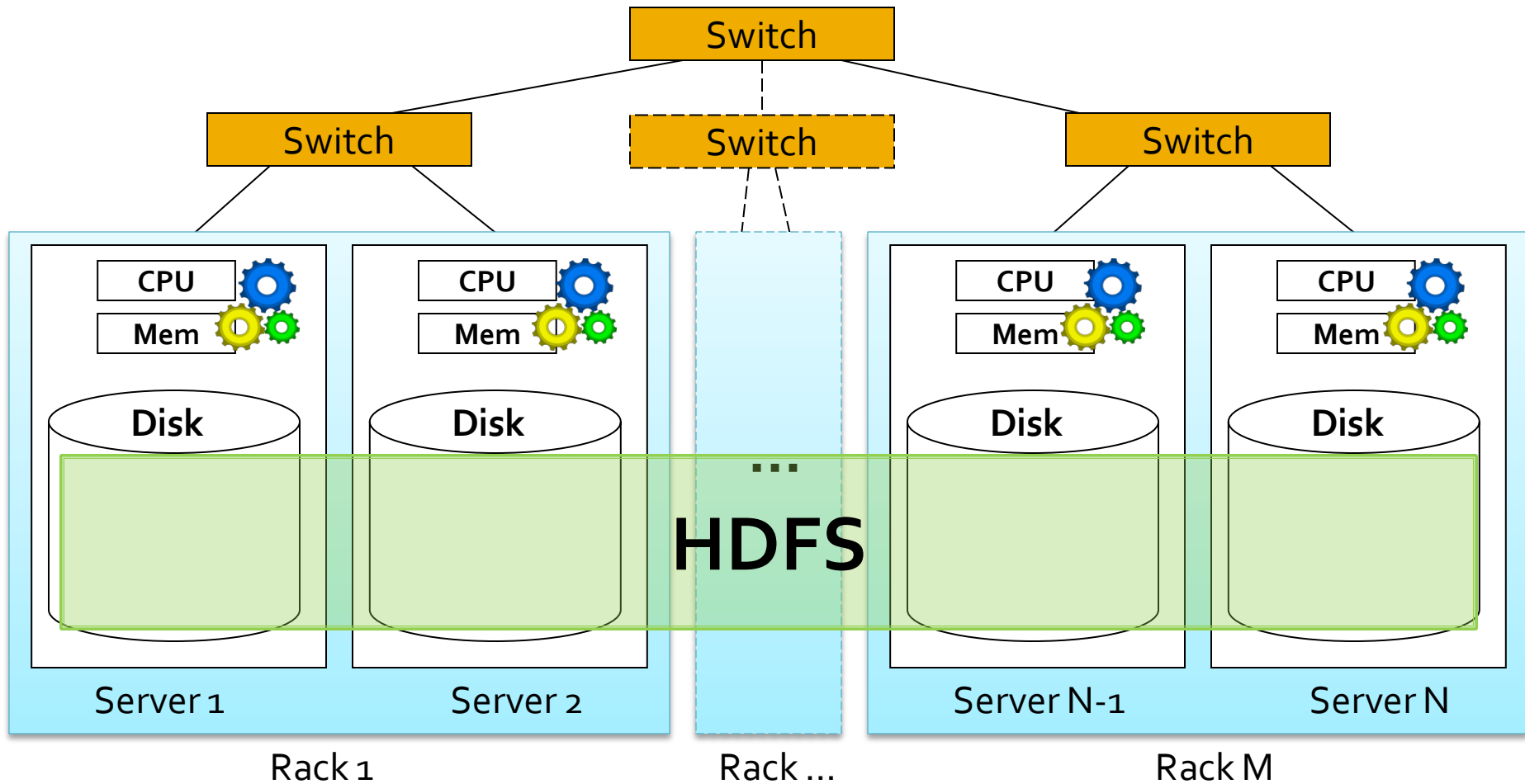
Example with number of replicas per chunk = 2

Hadoop: main components



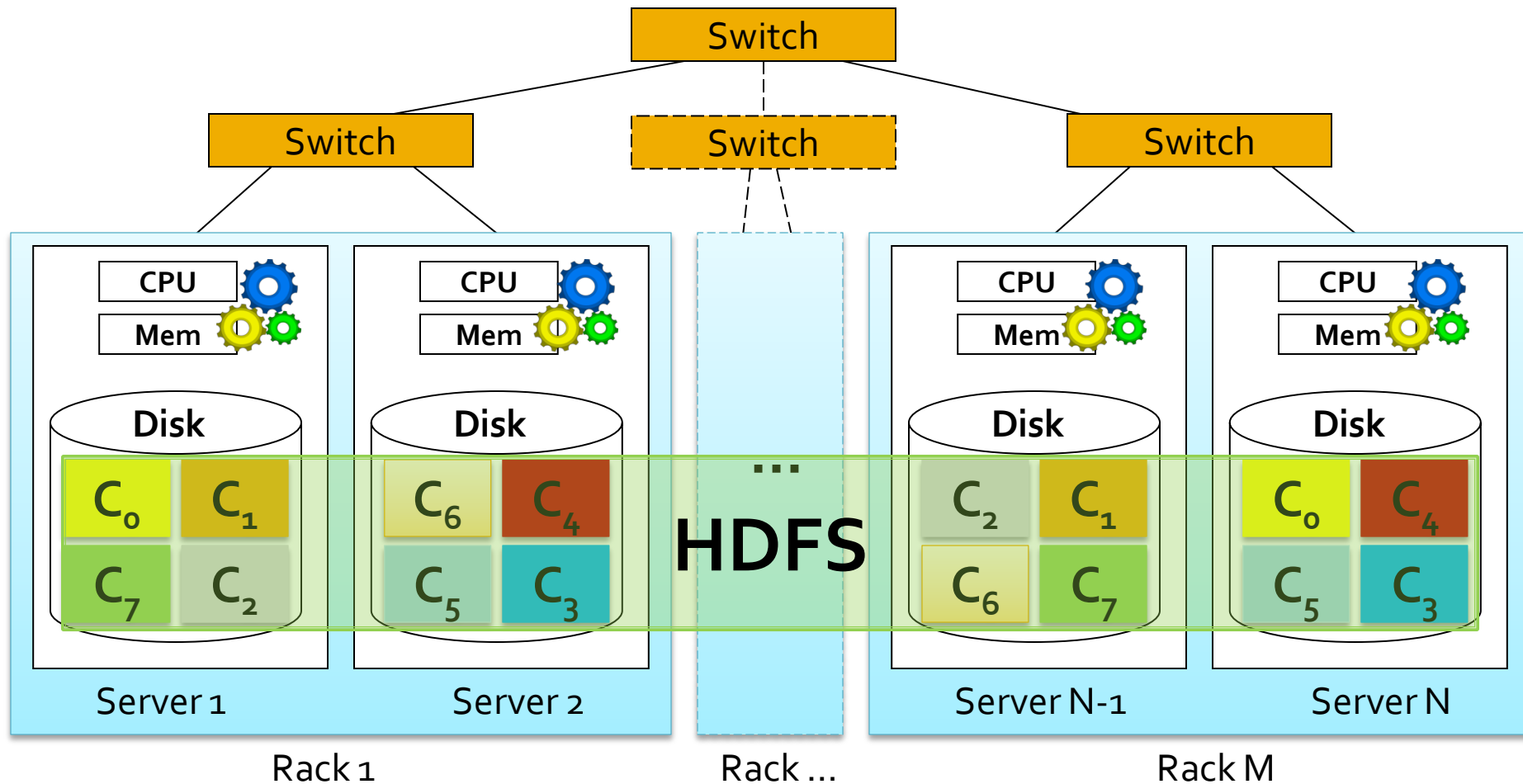
Example with number of replicas per chunk = 2

Hadoop: main components



Example with number of replicas per chunk = 2

Hadoop: main components



Example with number of replicas per chunk = 2

Distributed Big Data Processing Infrastructure

- Separates the **what** from the **how**
 - Hadoop programs are based on the MapReduce programming paradigm
 - MapReduce abstracts away the “distributed” part of the problem (scheduling, synchronization, etc)
 - Programmers focus on what
 - The distributed part (scheduling, synchronization, etc) of the problem is handled by the framework
 - The Hadoop infrastructure focuses on how

Distributed Big Data Processing Infrastructure

- But an in-depth knowledge of the Hadoop framework is important to develop efficient applications
 - The design of the application must exploit data locality and limit network usage/data sharing

HDFS

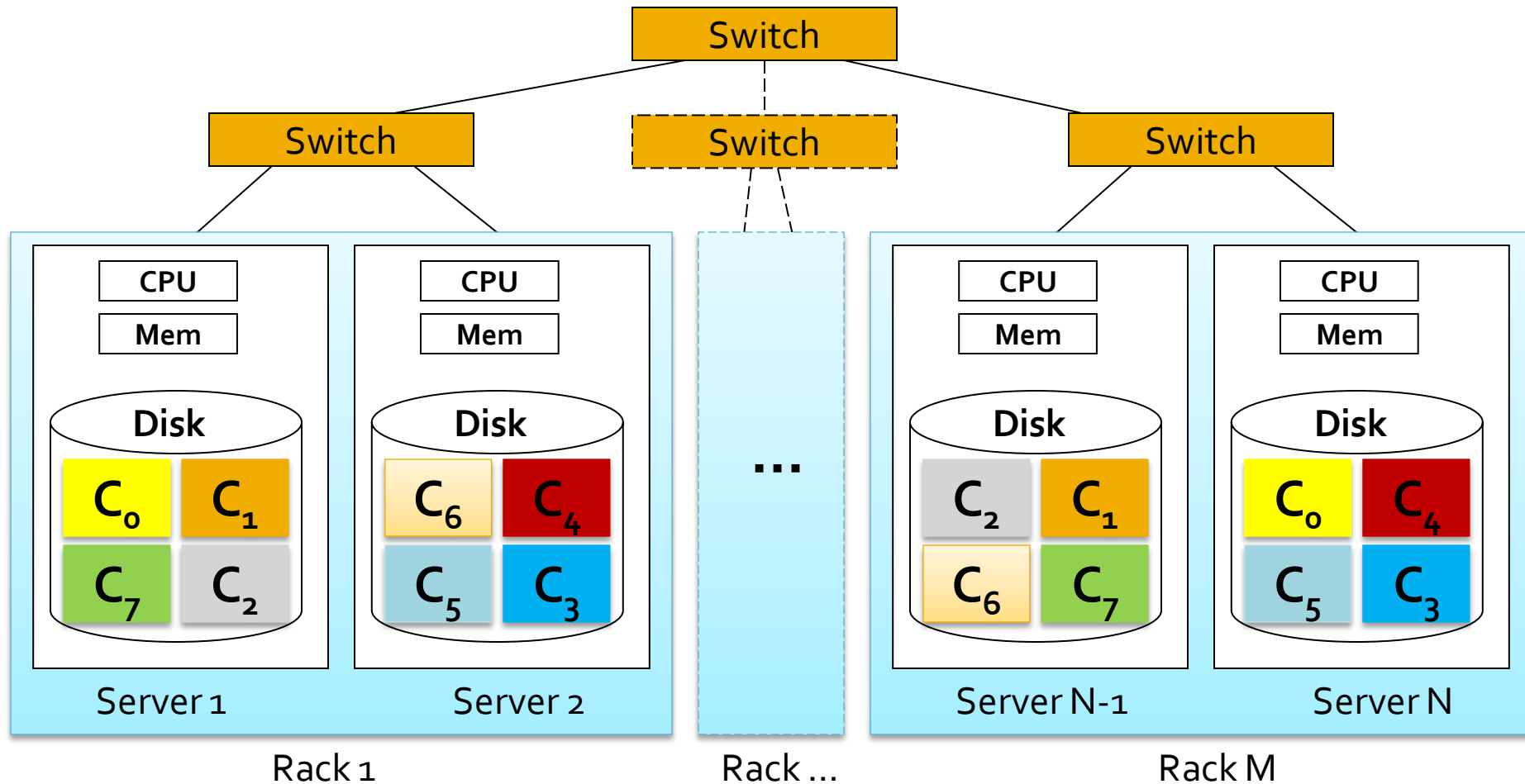
- HDFS
 - Standard Apache Hadoop distributed file system
 - Provides global file namespace
 - Stores data redundantly on multiple nodes to provide persistence and availability
 - Fault-tolerant file system
- Typical usage pattern
 - Huge files (GB to TB)
 - Data is rarely updated
 - Reads and appends are common
 - Usually, random read/write operations are not performed

HDFS (Hadoop File System)

- Each file is split in “chunks/blocks” that are spread across the servers
 - Each chunk is replicated on different servers (usually there are 3 replicas per chunk)
 - Ensures persistence and availability
 - To increase persistence and availability, replicas are stored in different racks, if it is possible
 - Each chunk/block contains a part of the content of one single file
 - You cannot have the content of two files in the same chunk/block
 - Typically each chunk is 64-128MB

HDFS

We have a maximum number of $N-1$ failure manageable at the same time, where N is the number of server



Example with number of replicas per chunk = 2

HDFS

- The Master node, a.k.a. Name Nodes in HDFS, is a special node/server that
 - Stores HDFS metadata
 - E.g., the mapping between the name of a file and the location of its chunks
 - Might be replicated
- Client applications: file access through HDFS APIs
 - Talk to the master node to find data/chuck servers associated with the file of interest
 - Connect to the selected chunk servers to access data

Hadoop ecosystem

- Many Hadoop-related projects/systems are available
 - Hive
 - A distributed relational database, based on MapReduce, for querying data stored in HDFS by means of a query language based on SQL
 - HBase
 - A distributed column-oriented database that uses HDFS for storing data
 - Pig
 - A data flow language and execution environment, based on MapReduce, for exploring very large datasets

Hadoop ecosystem

- Sqoop
 - A tool for efficiently moving data from traditional relational databases and external flat file sources to HDFS
- ZooKeeper
 - A distributed coordination service. It provides primitives such as distributed locks
-
- Each project/system addresses one specific class of problems

MapReduce: introduction

Warm up: Word Count

- Input
 - A large textual file of words
- Problem
 - Count the number of times each distinct word appears in the file
- Output
 - A list of pairs <word, number>, counting the number of occurrences of each specific word in the input file

Word Count

- Case 1: Entire file fits in main memory

Word Count

- Case 1: Entire file fits in main memory
 - A traditional single node approach is probably the most efficient solution in this case
 - The complexity and overheads of a distributed system affects the performance when files are “small”
 - “small” depends on the resources you have

Word Count

- Case 1: Entire file fits in main memory
 - A traditional single node approach is probably the most efficient solution in this case
 - The complexity and overheads of a distributed system affects the performance when files are “small”
 - “small” depends on the resources you have
- Case 2: File too large to fit in main memory

Word Count

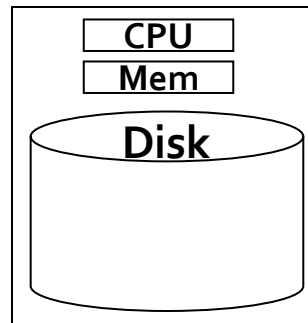
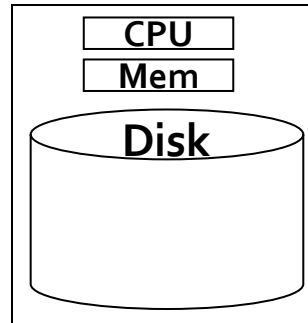
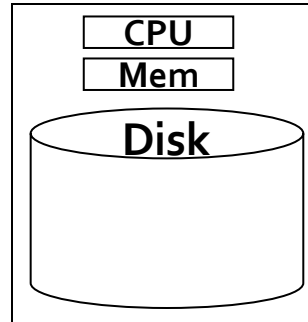
- Case 1: Entire file fits in main memory
 - A traditional single node approach is probably the most efficient solution in this case
 - The complexity and overheads of a distributed system affects the performance when files are “small”
 - “small” depends on the resources you have
- Case 2: File too large to fit in main memory
 - How can we split this problem in a set of (almost) **independent sub-tasks**, and
 - execute them **in parallel** on a cluster of servers?

Word Count with a very large file

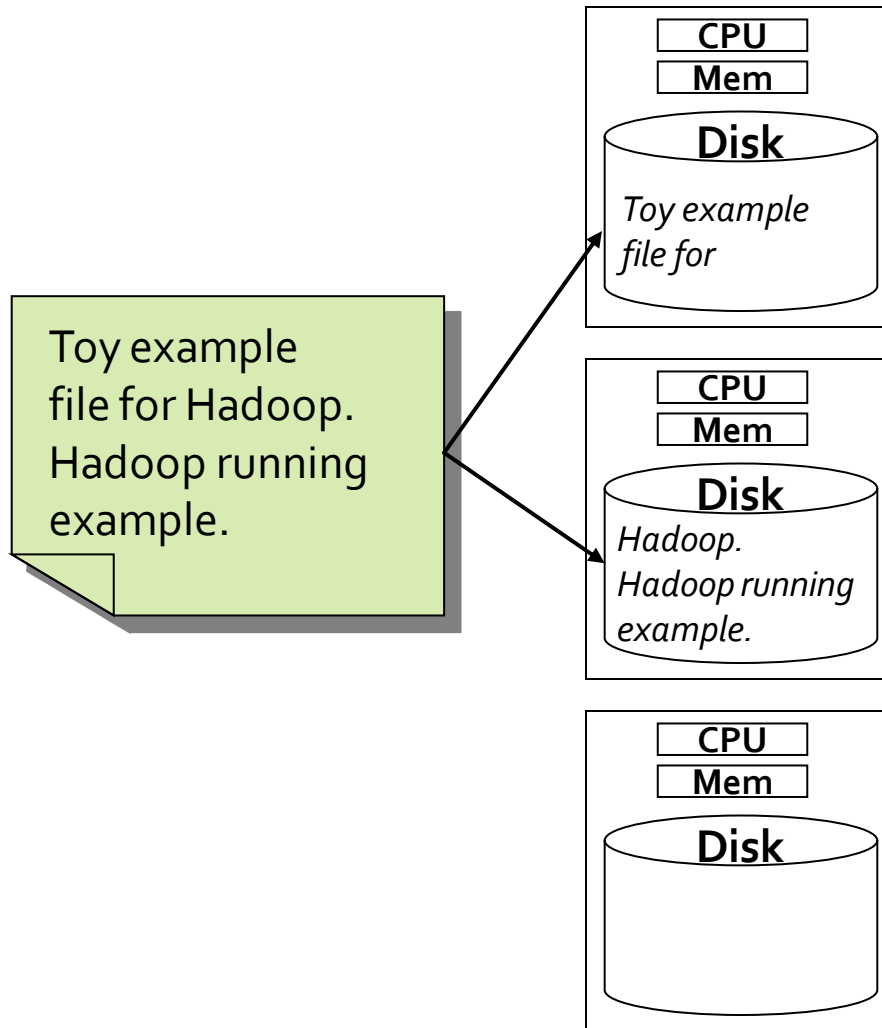
- Suppose that
 - The cluster has **3 servers**
 - The content of the input file is
 - “Toy example file for Hadoop. Hadoop running example.”
 - The input file is split into **2 chunks**
 - The number of **replicas** is **1**

Word Count with a very large file

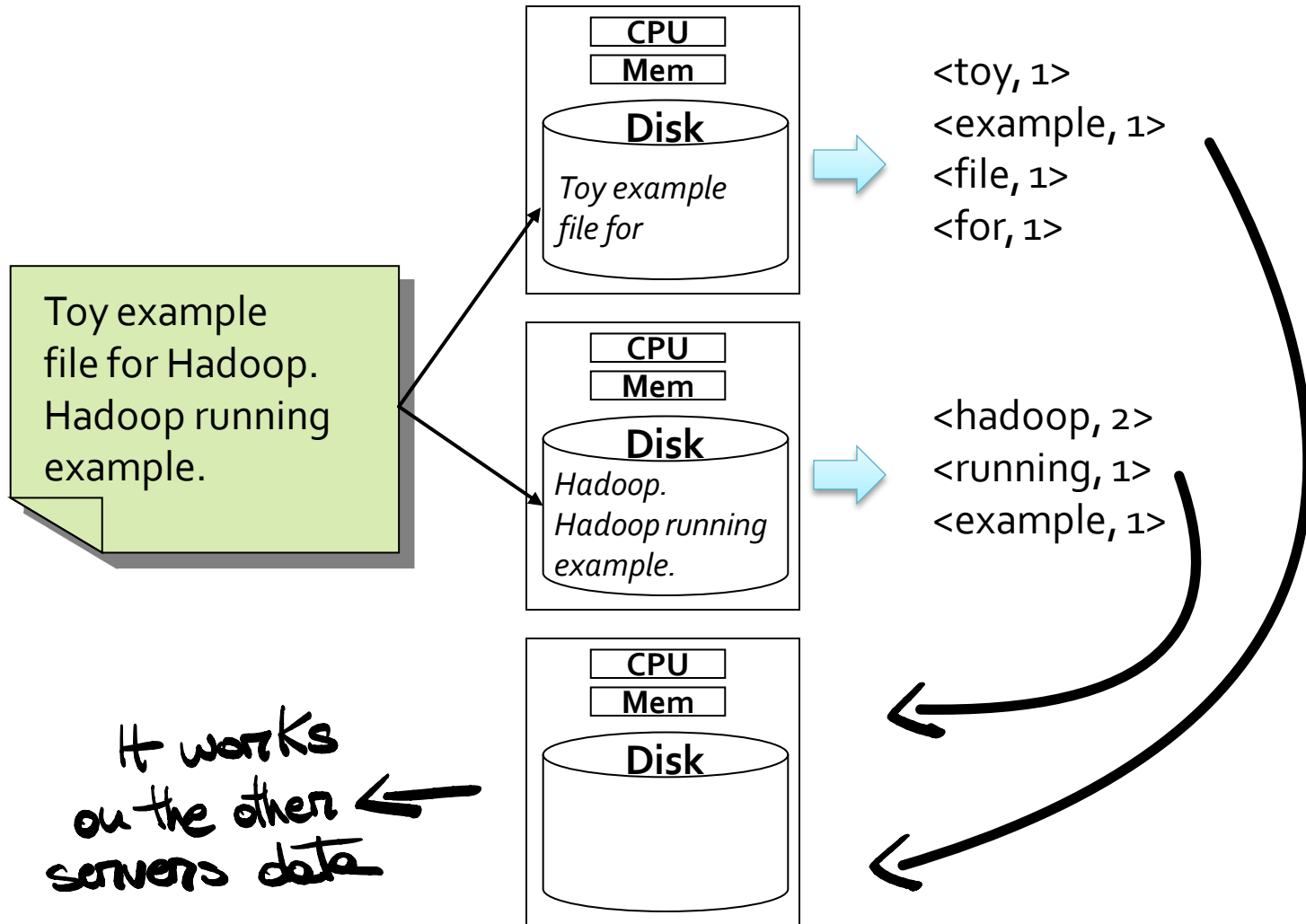
Toy example
file for Hadoop.
Hadoop running
example.



Word Count with a very large file



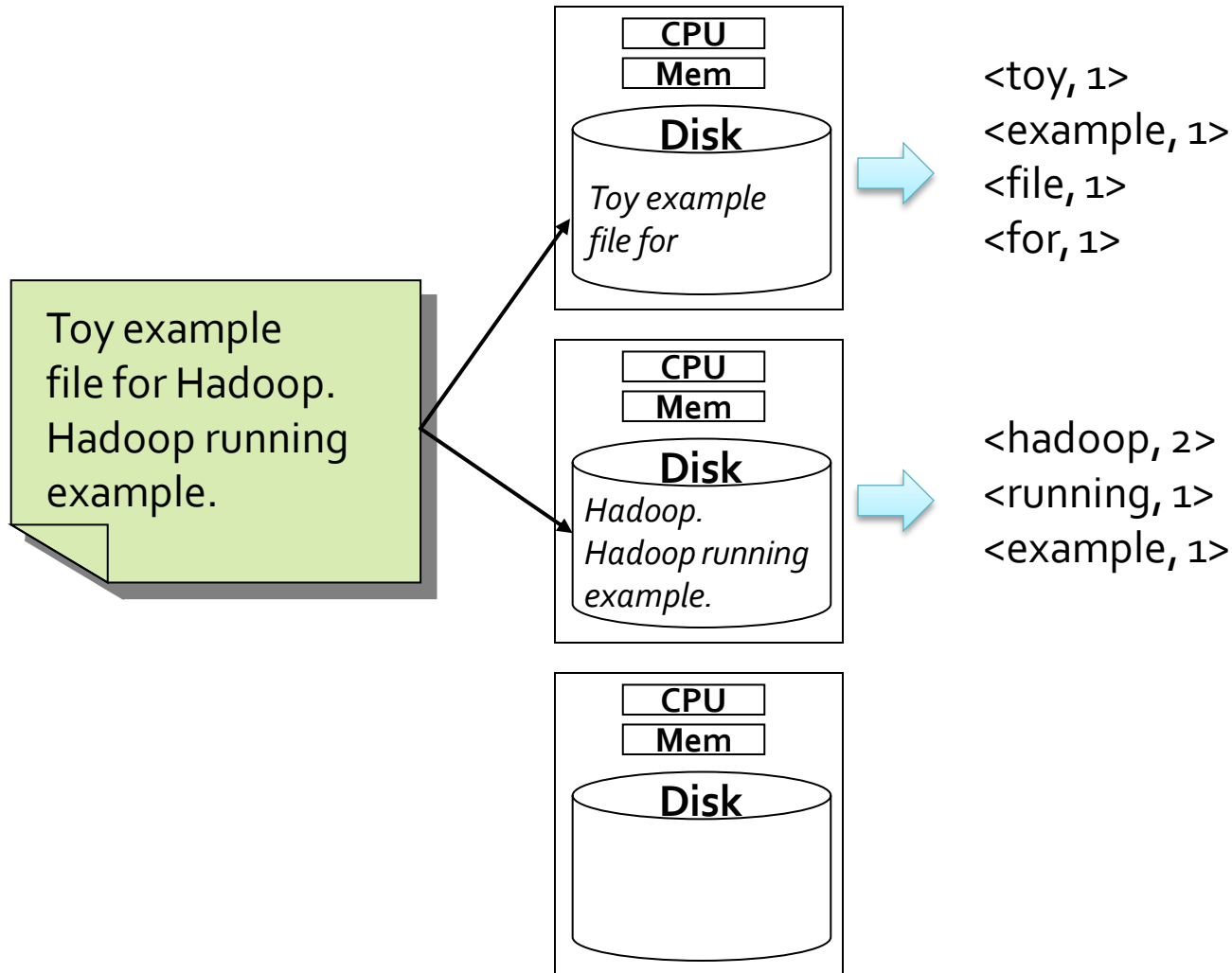
Word Count with a very large file



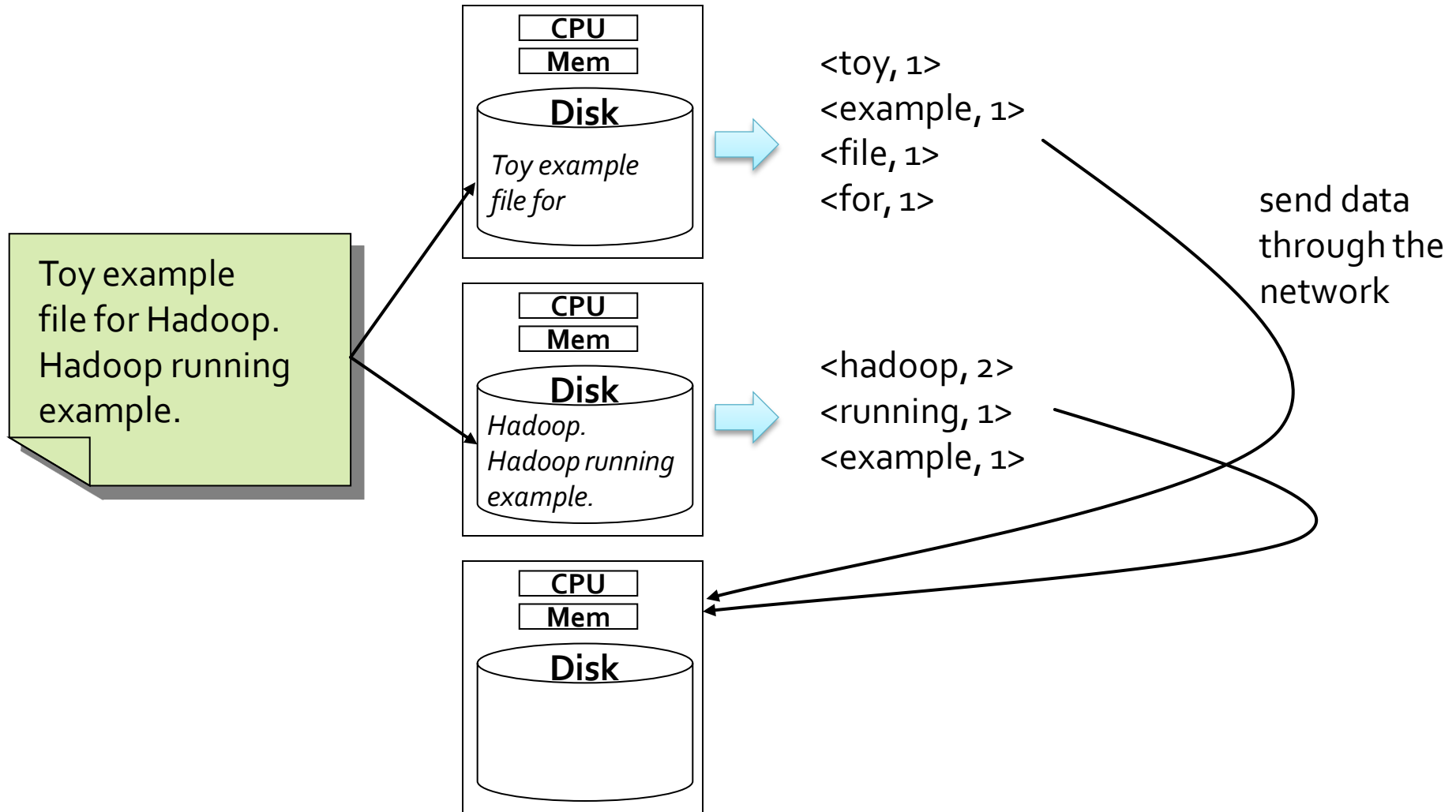
Word Count with a very large file

- The problem can be easily parallelized
 1. Each server processes its chunk of data and counts the number of times each word appears in its own chunk
 - Each server can execute its sub-task independently from the other servers of the cluster
→ synchronization is not needed in this phase
 - The output generated from each chunk by each server represents a partial result

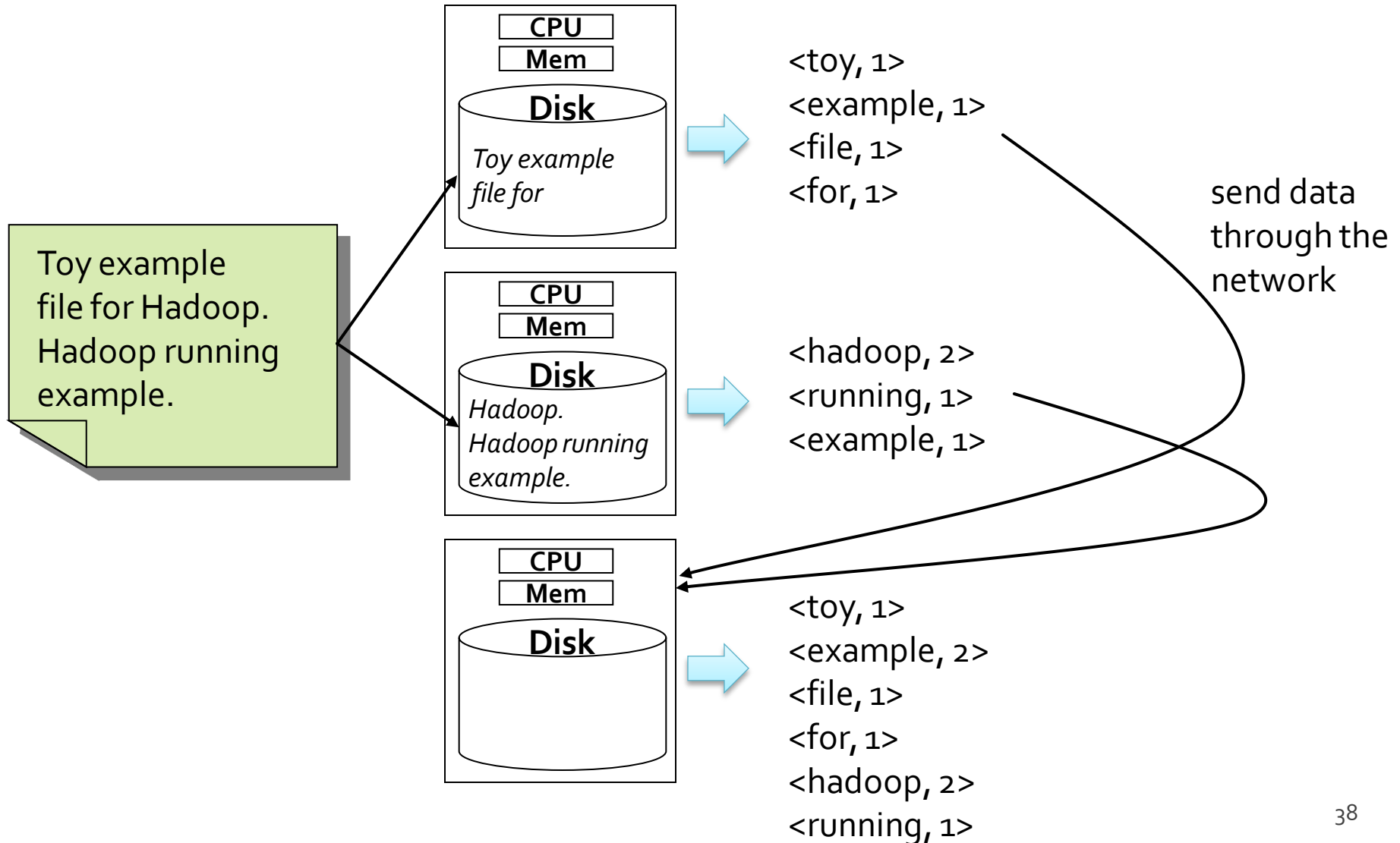
Word Count with a very large file



Word Count with a very large file



Word Count with a very large file



Word Count with a very large file

2. Each server sends its local (partial) list of pairs **<word, number of occurrences in its chunk>** to a server that is in charge of **aggregating** all local results and computing the global result
 - The server in charge of computing the global result needs to receive all the local (partial) results to compute and emit the final list
 - A synchronization operation is needed in this phase

Word Count: a more realistic example

- Case 2: File too large to fit in main memory
- Suppose that
 - The file size is 100 GB and the number of distinct words occurring in it is at most 1,000
 - The cluster has 101 servers
 - The file is spread across 100 servers and each of these servers contains one (different) chunk of the input file
 - i.e., the file is optimally spread across 100 servers (each server contains $1/100$ of the file in its local hard drives)

Word Count: complexity

- Each server reads 1 GB of data from its local hard drive (it reads one chunk from HDFS)
 - Few seconds
- Each local list consists of at most 1,000 pairs (because the number of distinct words is 1,000)
 - Few MBs
- The maximum amount of data sent on the network is $100 \times$ size of local list (number of servers $\times$ local list size)
 - Some MBs

Word Count: scalability

- We can define scalability along two dimensions
 - In terms of data:
 - Given twice the amount of data, the word count algorithm takes approximately no more than twice as long to run
 - Each server processes $2 \times$ data $\Rightarrow 2 \times$ execution time to compute local list
 - In terms of resources
 - Given twice the number of servers, the word count algorithm takes approximately no more than half as long to run
 - Each server processes $\frac{1}{2} \times$ data $\Rightarrow \frac{1}{2} \times$ execution time to compute local list

Word Count: scalability

- The time needed to send local results to the node in charge of computing the final result and the computation of the final result are considered negligible in this running example
- Frequently, this assumption is not true
 - It depends
 - on the complexity of the problem
 - on the ability of the developer to limit the amount of data sent on the network

MapReduce-approach key ideas

- Scale “out”, not “up”
 - Increase the number of servers, avoiding to upgrade the resources (CPU, memory) of the current ones
- Move processing to data
 - The network has a limited bandwidth
- Process data sequentially, avoid random access
 - Seek operations are expensive
 - Big data applications usually read and analyze all input records/objects
 - Random access is useless

Data locality

- Traditional distributed systems (e.g., HPC) move data to computing nodes (servers)
 - This approach cannot be used to process TBs of data
 - The network bandwidth is limited
- Hadoop moves code to data
 - Code (few KB) is copied and executed on the servers where the chunks of data are stored
 - This approach is based on “data locality”

Hadoop and MapReduce

- Hadoop/MapReduce is designed for
 - Batch processing involving (mostly) full scans of the input data *↳ Don't use Hadoop/Map Reduce for stream of data*
 - Data-intensive applications
 - Read and process the whole Web (e.g., PageRank computation) *↳ It assign a rank to web pages*
 - Read and process the whole Social Graph (e.g., LinkPrediction, a.k.a. "friend suggestion")
 - Log analysis (e.g., Network traces, Smart-meter data, ..)

Hadoop and MapReduce

- Hadoop/MapReduce is not the panacea for all Big Data problems

- Hadoop/MapReduce does not ^{fit}  well

- Iterative problems
- Recursive problems
- Stream data processing
- Real-time processing

Many machine learning algorithms cannot use Hadoop/MapReduce because they need a global view of the data to take a decision

The MapReduce Programming Paradigm

MapReduce and Functional programming

- The MapReduce programming paradigm is based on the basic concepts of Functional programming
- MapReduce “implements” a subset of functional programming
 - The programming model appears quite limited and strict
 - Everything is based on two “functions” with predefined signatures
 - Map and Reduce

What can we do with MapReduce?

- Solving complex problems is difficult
- However, there are several important problems that can be adapted to MapReduce
 - Log analysis
 - PageRank computation
 - Social graph analysis
 - Sensor data analysis
 - Smart-city data analysis
 - Network capture analysis

Building blocks: Map and Reduce

- MapReduce is based on two main “building blocks”
 - **Map** and **Reduce** functions
- **Map function**
 - It is **applied over each element** of an input data set and **emits** a set of **(key, value) pairs**
- **Reduce function**
 - It is **applied over each set of (key, value) pairs** (emitted by the map function) **with the same key** and **emits** a set of **(key, value) pairs** → Final result

Word count running example

- Input
 - A textual file (i.e., a list of words)
- Problem
 - Count the number of times each distinct word appears in the file
- Output
 - A list of pairs <word, number of occurrences in the input file>

Word count running example

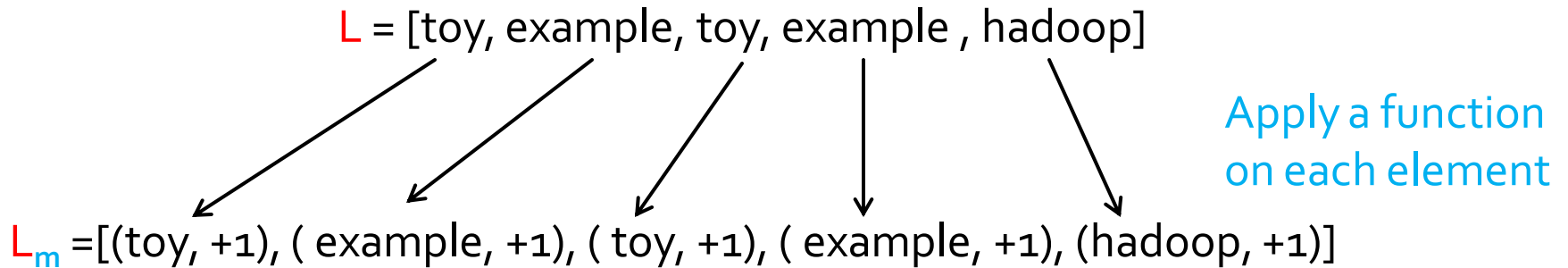
- The input textual file is considered as a list of words **L**

Word count running example

L = [toy, example, toy, example , hadoop]

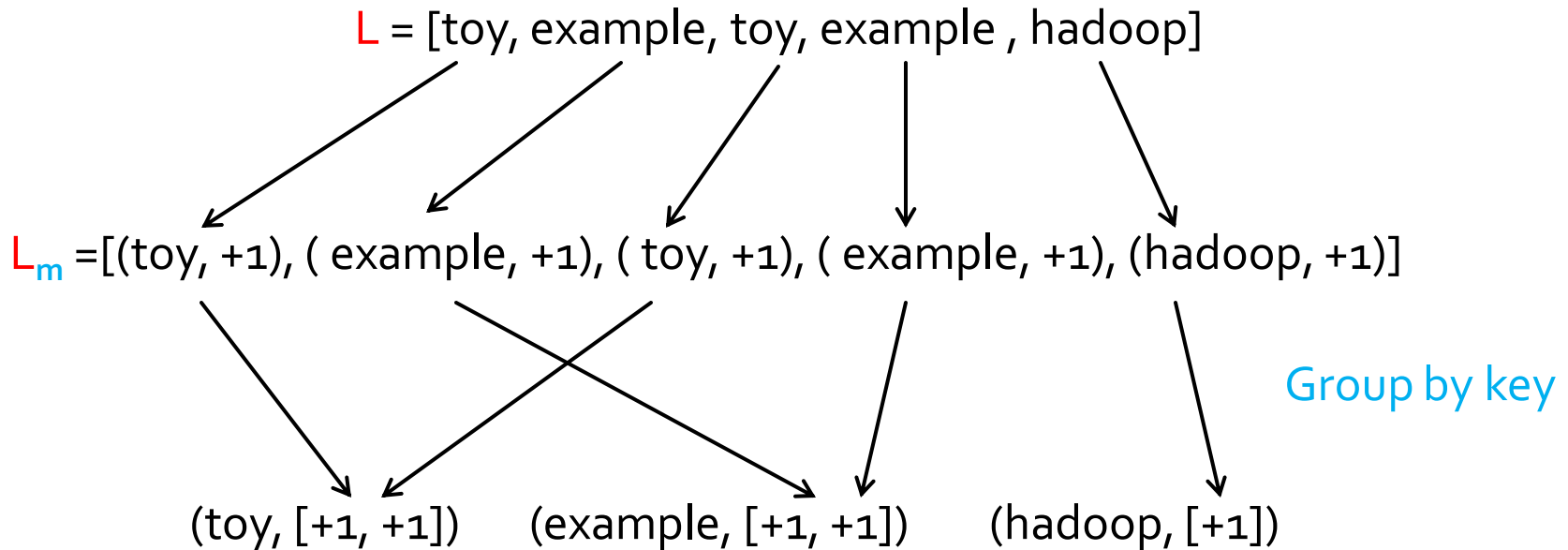
[...] denotes a list. (k, v) denotes a key-value pair.

Word count running example

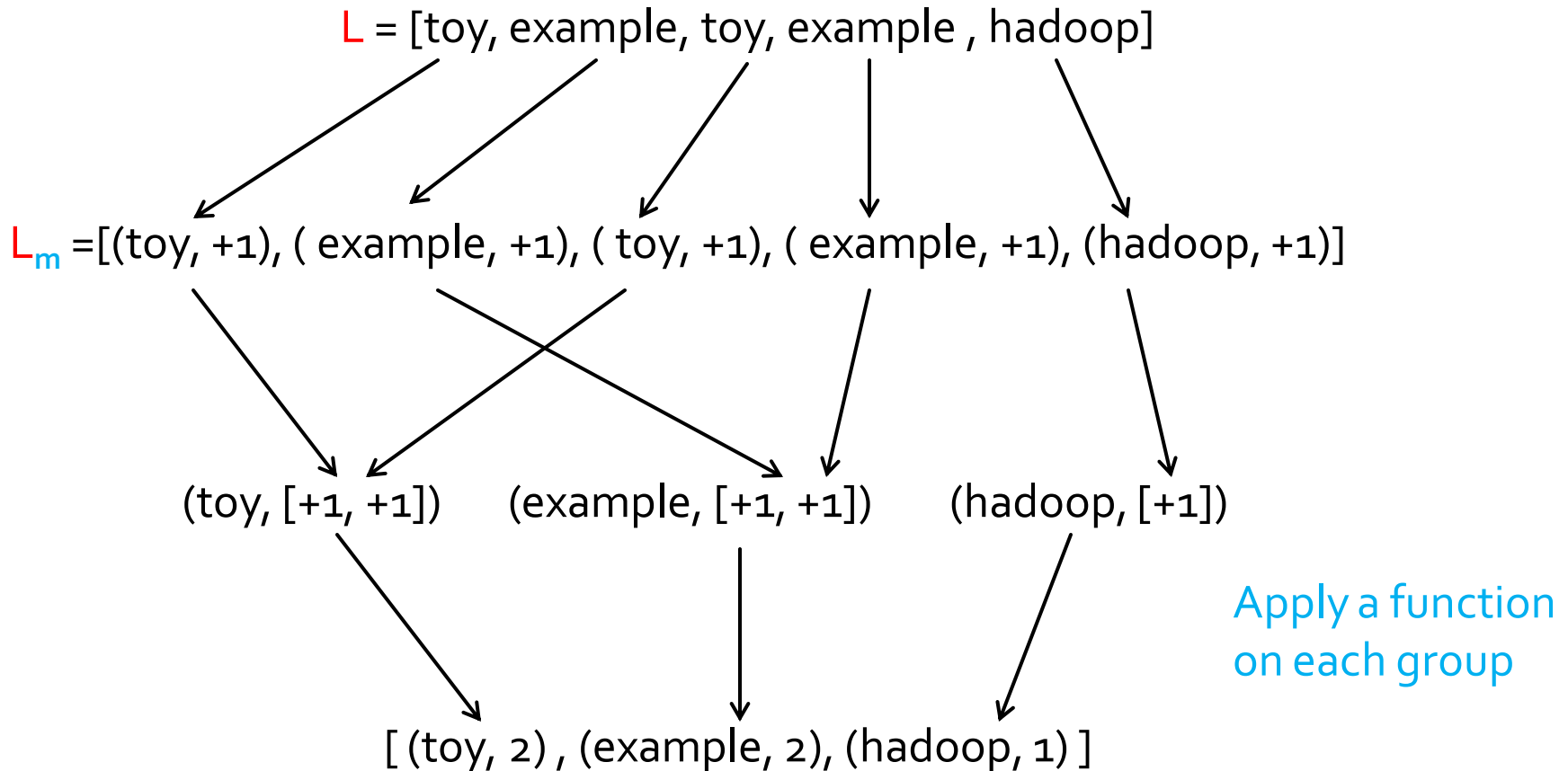


[...] denotes a set. (k, v) denotes a key-value pair.

Word count running example

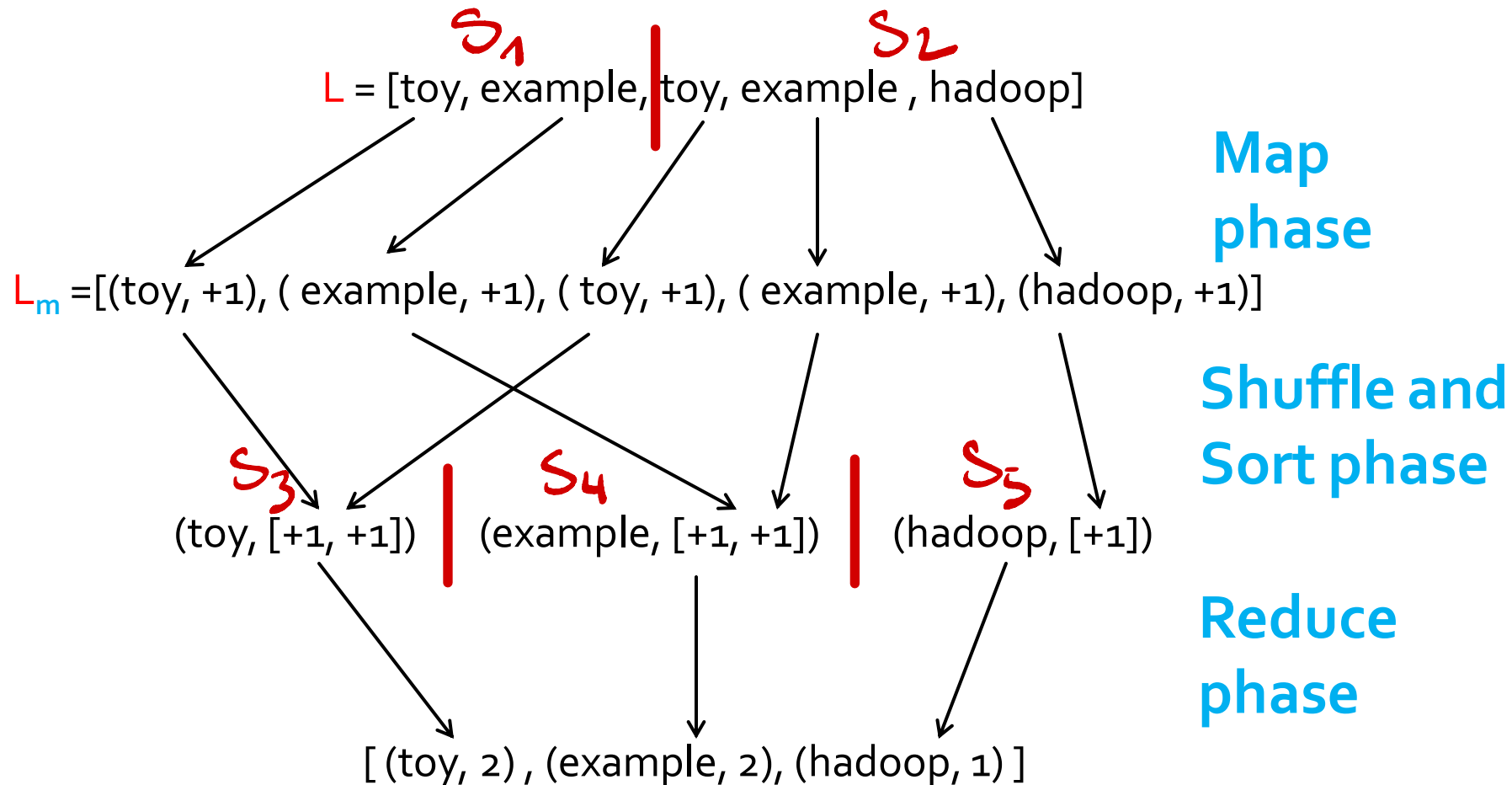


Word count running example



[...] denotes a list. (k, v) denotes a key-value pair.

Word count running example



[...] denotes a list. (k, v) denotes a key-value pair.

Word count running example

- The input textual file is considered as a list of words L
- A key-value pair $(w, 1)$ is emitted for each word w in L
 - i.e., the **map** function is
$$m(w) = (w, 1)$$
 - A new list of (key, value) pairs L_m is generated

Word count running example

- The key-value pairs in L_m are aggregated by key (i.e., by word w in our example)
 - One group G_w is generated for each word w
 - Each group G_w is a key-list pair (w , [list of values]) where [list of values] contains all the values of the pairs associated with the word w
 - i.e., [list of values] is a list of [1, 1, 1, ...] in our example
 - Given a group G_w , the number of ones [1, 1, 1, ...] is equal to the occurrences of word w in the input file

Word count running example

- A key-value pair (w , $\text{sum } G_w[\text{list of values}]$) is emitted for each group G_w
 - i.e., the **reduce** function is
$$r(G_w) = (w, \text{sum}(G_w[\text{list of values}]))$$
- The list of emitted pairs is the result of the word count problem
 - One pair (word w , num. of occurrences) for each word in our running example

MapReduce: Map

- The **Map** phase can be viewed as a **transformation** over each element of a data set
 - This transformation is a function **m** defined by developers
 - **m** is invoked one time for each input element
 - Each invocation of **m** happens in **isolation**
 - The application of **m** to each element of a data set can be parallelized in a straightforward manner

MapReduce: Reduce

- The **Reduce** phase can be viewed as an **aggregate** operation
 - The aggregate function is a function **r** defined by developers
 - **r** is **invoked one time for each distinct key** and aggregates all the values associated with it
 - Also the reduce phase can be performed in parallel and in isolation
 - Each group of key-value pairs with the same key can be processed in isolation

MapReduce: Shuffle and Sort

- The shuffle and sort phase is always the same
 - i.e., group the output of the map phase by key
 - It does not need to be defined by developers
 - It is already provided by the Hadoop system

Data Structures

- Key-value pair is the basic data structure in MapReduce
 - Keys and values can be: integers, float, strings, ...
 - They can also be (almost) arbitrary data structures defined by the designer
- Both input and output of a MapReduce program are lists of key-value pairs
 - Note that also the input is a list of key-value pairs

Data Structures

- The design of MapReduce involves
 - Imposing the key-value structure on the input and output data sets
 - E.g., for a collection of Web pages, input keys may be URLs and values may be their HTML content

Formal definition of Map and Reduce functions

- The map and reduce functions are formally defined as follows:
 - $\text{map}: (k_1, v_1) \rightarrow [(k_2, v_2)]$
 - $\text{reduce}: (k_2, [v_2]) \rightarrow [(k_3, v_3)]$
- Since the input data set is a list of key-value pairs, the argument of the map function is a key-value pair

Formal definition of Map and Reduce functions

- Map function
 - $\text{map}: (k_1, v_1) \rightarrow [(k_2, v_2)]$
- The argument of the map function is a key-value pair
- Note that the map function
 - **Returns a list** of key-value pairs for each input pair
 - The list can be empty

Formal definition of Map and Reduce functions

- Reduce function
 - $\text{reduce}: (k_2, [v_2]) \rightarrow [(k_3, v_3)]$
- Note that the reduce function
 - Is invoked once for each distinct key
 - Receives the complete list of values $[v_2]$ associated with a specific key k_2
 - Returns a list of key-value pairs for each input
 - The list can be empty

MapReduce Algorithms

- In many applications, the key part of the input data set is ignored
 - i.e., usually the map function does not consider the key of its key-value pair argument
 - E.g., word count problem
- Some specific applications exploit also the keys of the input data
 - E.g., keys can be used to uniquely identify records/objects

Word Count using MapReduce: Pseudocode

Input file: a textual document with one word per line
The map function is invoked over each word of the input file

```
map(key, value):  
    // key: offset of the word in the file  
    // value: a word of the input document  
    emit(value, 1)
```

```
reduce(key, values):  
    // key: a word; values: a list of integers  
    occurrences = 0  
    for each c in values:  
        occurrences = occurrences + c  
  
    emit(key, occurrences)
```